方向A：基础协议与主机处理

A1

如果`self.conn_type="my_db"`，初始URI 前缀如何构造?

A. `my_db://`
B. `my-db://`
C. `MY_DB://`
D. `//my_db`

A2

如果`self.host`中包含协议，且该协议与`self.conn_type`不同，代码会如何处理?

A. 忽略host 中的协议
B. 将host 中的协议保存为`protocol_to_add`，并使用去掉协议后的host
C. 直接抛出异常
D. 将conn_type 覆盖为host 中的协议

A3

如果`self.host`中不包含`"://"`，会发生什么?

A. `host_to_use`使用原始host，`protocol_to_add`为`None`
B. host 被设置为空字符串
C. 必须从conn_type 中重新生成host
D. 抛出解析异常

方向B：认证信息与主机块组装

B1

如果login 和password 都存在，认证信息会被组装成什么形式?

A. `login@password`
B. `login:password@`
C. `password:login@`
D. `@login/password`

B2

如果存在端口，但host 和认证信息都为空，代码会向`host_block`添加什么?

A. `":端口"`
B. `"@:端口"`
C. `"//端口"`
D. `"?port=端口"`

B3

如果存在schema，代码会如何拼接？

A. 添加到URI query 中
B. 作为经过转义的路径片段追加到`host_block`
C. 覆盖host
D. 作为认证信息追加到`authority_block`

方向C：扩展参数与fallback

C1

在什么情况下`extra_dict`可以直接展开为普通query 参数？

A.`extra_dict`中所有值都是字符串，并且编码后再解析能还原为原字典
B. 只要`self.extra`非空
C. 只要存在schema
D. 只要`urlencode`抛出异常

C2

如果扩展参数不能安全展开，代码会如何处理？

A. 丢弃扩展参数
B. 将原始`self.extra`放入`EXTRA_KEY`后再编码
C. 抛出异常
D. 将extra 追加到host 中

C3

扩展query 的拼接前缀由什么决定？

A. 是否存在login
B. 是否存在password
C. 是否存在schema：有schema 时使用`"?"`，否则使用`"/?"`
D. 是否存在端口

总览题

T1

下面哪项最准确地概括该函数？

A. 只负责拼接主机名和端口
B. 按阶段组装协议、可选认证信息、主机块、schema 和扩展query，并对无法直接展开的extra 使用fallback 编码
C. 只检查URI 是否符合RFC3986
D. 只负责处理Airflow 日志